

Predicting Credit Defaults Using Imbalanced Data Techniques

Alex Ojemann ojema001@umn.edu	Dehkontee Chea Cuppah cuppa001@umn.edu
----------------------------------	---

Introduction

This project explores the challenges and solutions associated with predicting credit default risks using a highly imbalanced dataset. The objective is to leverage machine learning models and data preprocessing techniques to address the class imbalance and improve predictive performance. We utilize methods specific to imbalanced data such as Synthetic Minority Oversampling Technique (SMOTE), Adaptive Synthetic Sampling Approach (ADASYN), and the use of evaluation metrics such as F2 score. We show that most of our models perform much better when trained on the samples that include synthetic samples of the minority class generated using SMOTE or ADASYN. Of the models we explore, we determine that a Hybrid Naive Bayes Classifier using the SMOTE or ADASYN data and a high smoothing parameter is the best fit for the data through analysis of F2 scores and confusion matrices. The project ultimately demonstrates how machine learning can assist financial institutions in mitigating credit risks.

Dataset Used

The application.csv dataset from Kaggle serves as the core resource for this project, providing extensive details about loan applicants. This dataset contains 307,511 records and 122 features, capturing a wide array of demographic, financial, and credit-related attributes essential for building machine learning models. Below is a detailed overview of the dataset:

1. Key Identifiers and Target Variable

- SK_ID_CURR: A unique numeric identifier for each applicant. This was not included as a feature in any of our models.

- TARGET: Binary target variable where `1` indicates default and `0` indicates non-default. This is the primary label for prediction.

2. Demographic Attributes

- CODE_GENDER: Gender of the applicant ('Male' or 'Female').
- CNT_CHILDREN: Number of children dependent on the applicant.
- DAYS_BIRTH: Applicant's age in days (negative values represent the number of days since birth).

3. Financial Attributes

- AMT_INCOME_TOTAL: Total annual income of the applicant.
- AMT_CREDIT: Total credit amount requested.
- AMT_ANNUITY: Annual loan repayment amount.
- AMT_GOODS_PRICE: Price of the goods associated with the loan application.

4. Employment and Occupation Details

- DAYS_EMPLOYED: Number of days the applicant has been employed. Notable for containing some extreme values requiring preprocessing.
- OCCUPATION_TYPE: Job type or industry classification.
- ORGANIZATION_TYPE: The type of organization employing the applicant.

5. Housing and Living Conditions

- FLAG_OWN_CAR: Indicates whether the applicant owns a car ('Yes' or 'No').
- FLAG_OWN_REALTY: Indicates property ownership.
- NAME_HOUSING_TYPE: Specifies the type of housing, such as owned apartments or rented accommodations.

6. Credit History Features

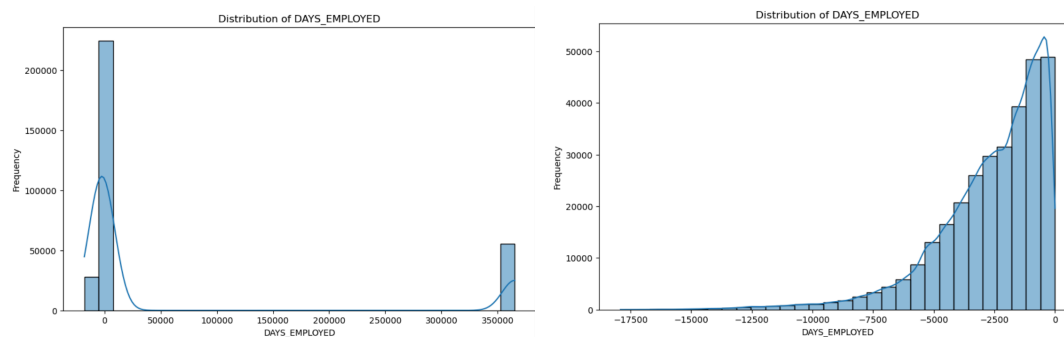
- AMT_REQ_CREDIT_BUREAU_X: Frequency of credit bureau inquiries over various time periods, such as a day, week, month, quarter, or year.

7. Documentation and Verification

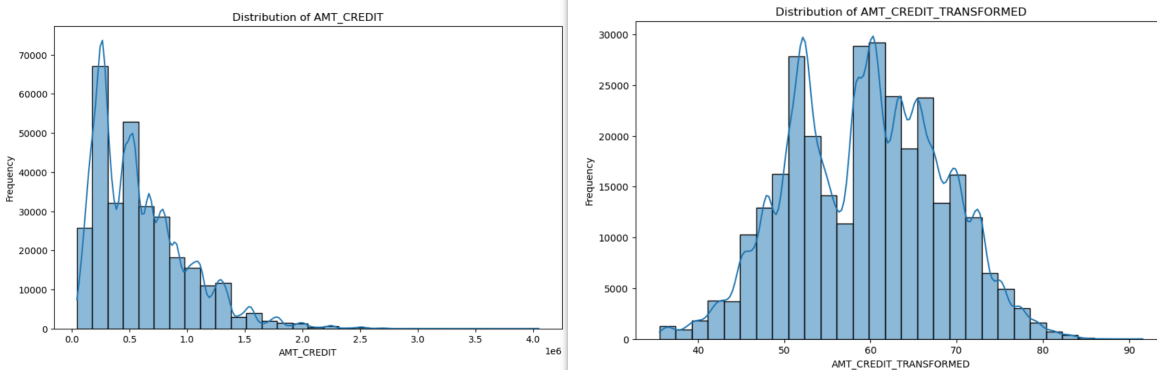
- FLAG_DOCUMENT_X (X = 2 to 21): Flags indicating whether the applicant provided specific documents required for loan approval.

Feature Engineering

The feature engineering for this project involved the imputation of values that did not make sense in context, the transformation of continuous features that were heavily skewed, and the scaling of all continuous features using a standard scaler. There was one feature in particular that had values that did not make any sense in context, that being the number of days employed with many values of over 300,000. These values were set to null and imputed using a KNN imputer. The KNN imputer used the 10 nearest neighbors and only used a subset of the other features to fit the KNN, those being the number of days since birth, the number of days since changing their registration, the number of days since last changing their phone number, and the number of days since last changing their ID document because those are theoretically most related to the number of days employed. Below are the distributions of the number of days employed before and after the imputation.



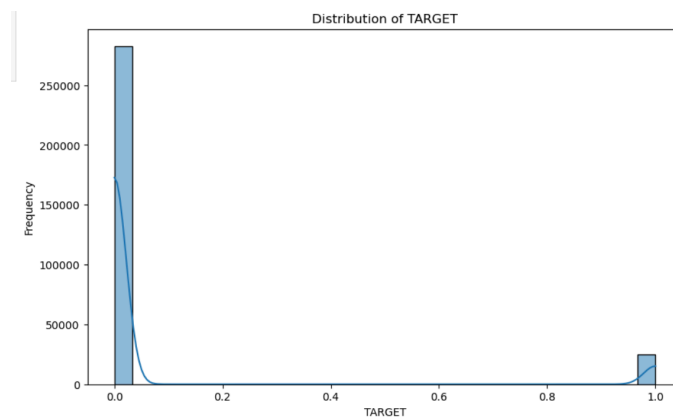
The next step was to transform skewed features. Most of the continuous features in the data set were skewed right, though some were close to normally distributed and others were skewed left. A Box-Cox transformation was conducted on those that were not close to normally distributed because this transformation is flexible enough to eliminate either left or right skew depending on which was exhibited by the feature. The Box-Cox transformation requires all values to be positive. The columns representing the number of days since an event were represented as negative numbers, but their interpreted significance is the same if the negative is dropped, so these features were multiplied by -1 before the transformation. The value of lambda was determined by the skew for each feature. These transformations were necessary because it is important for the features to be normally distributed for some of the models that we fit in a later section, particularly the naive bayes classifier. An example of the effect of the transformation for the feature representing the amount of credit is shown below.



Other feature engineering processes performed include scaling all continuous features using a standard scaler and one hot encoding text based features. This is necessary because some of the models that we fit in a later section require scaled features, especially the KNN model. There were not too many distinct values of the text based categorical features relative to the number of rows in the data set, so one hot encoding was deemed appropriate. Finally, three columns representing evaluations by external sources were removed due to very high volumes of null values.

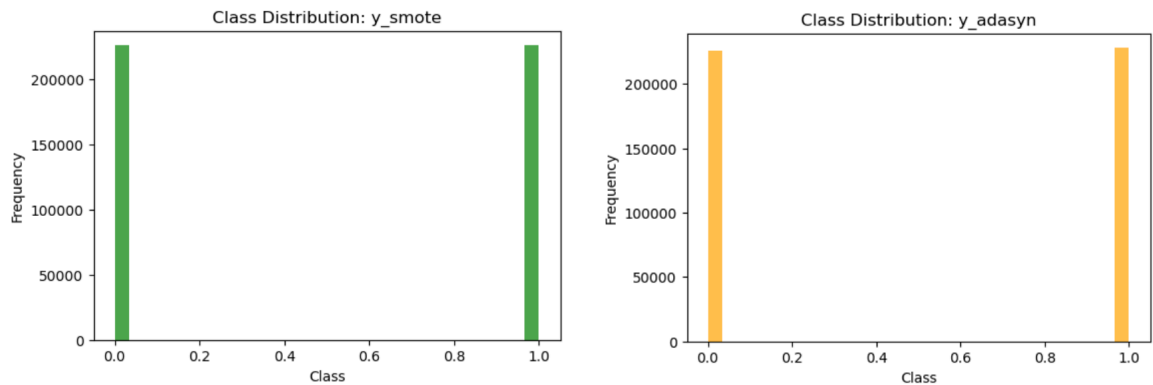
Sampling Methods

A critical consideration for this dataset is that the target classes, representing whether or not the borrower defaulted, are heavily imbalanced, with only a little over 8% of the borrowers having defaulted, as shown in the distribution below.



This imbalance requires the use of different techniques than those used for data with balanced target classes. One of these techniques is oversampling. We used the Synthetic Minority Oversampling Technique (SMOTE) and the Adaptive Synthetic Sampling Approach (ADASYN) to oversample the minority class, that in which the borrower defaulted, to achieve an equal distribution in the training data. These techniques both involve the generation of synthetic samples of the minority class by finding the 5 nearest neighbors of existing minority class samples and generating new samples that lie in between the sample and its neighbors. It is

critical to note that these techniques were not used for the testing data as that would result in improper evaluations of how the model would perform on unseen data in the real world. Below are the distributions of the target in the training data after SMOTE and ADASYN were applied respectively.



Evaluation Metrics

The evaluation metrics used for the models fit project were selected to best avoid the problems caused by an imbalanced dataset. Specifically, using accuracy as an evaluation metric for imbalanced data causes significant problems because models may have higher accuracies because they predict the majority class more often rather than because they add real predictive value. Precision and recall avoid this pitfall because in combination they do not reward a model for predicting the majority class more often. In the context of predicting credit defaults, recall is more important than precision because it is much more costly to loan money to someone who ends up defaulting on the loan than it is to not loan money to someone who would not default on the loan. For this reason, F2 score is used as our primary evaluation metric because it is a combination of precision and recall in which recall is given comparatively more weight. However, precision and recall are considered individually and confusion matrices are examined for each model for added context behind its F2 score.

Models and Results

The models trained for this project include a logistic regression model, a decision tree, a random forest, a KNN model, a Naive Bayes classifier, an AdaBoost model and an XGBoost model. These models were each trained on the unaltered, SMOTE, and ADASYN training data and the precision, recall, and F2 score of each model on the test set were compared in the tables below. Default hyperparameters were used for each model in this comparison.

Precision	Original Training Data	SMOTE Training Data	ADASYN Training Data
-----------	------------------------	---------------------	----------------------

Logistic Regression	0	0.3333	0.2
Decision Tree	0.1142	0.1061	0.1156
Random Forest	0.6667	0.3592	0.3571
Gaussian Naive Bayes	0.0821	0.0985	0.0815
KNN	0.1339	0.1107	0.1086
AdaBoost	0	0.1762	0.2021
XGBoost	0.2941	0.4571	0.1795

Recall	Original Training Data	SMOTE Training Data	ADASYN Training Data
Logistic Regression	0	0.0004	0.0002
Decision Tree	0.1360	0.1388	0.1494
Random Forest	0.0004	0.0075	0.007
Gaussian Naive Bayes	0.9843	0.9901	0.9934
KNN	0.0147	0.2699	0.2695
AdaBoost	0	0.0598	0.0612
XGBoost	0.0030	0.0032	0.0014

F2 Score	Original Training Data	SMOTE Training Data	ADASYN Training Data
Logistic Regression	0	0.0005	0.0003
Decision Tree	0.1310	0.1307	0.1412
Random Forest	0.0005	0.0093	0.0088
Gaussian Naive Bayes	0.3077	0.3066	0.3068
KNN	0.0179	0.2096	0.2079

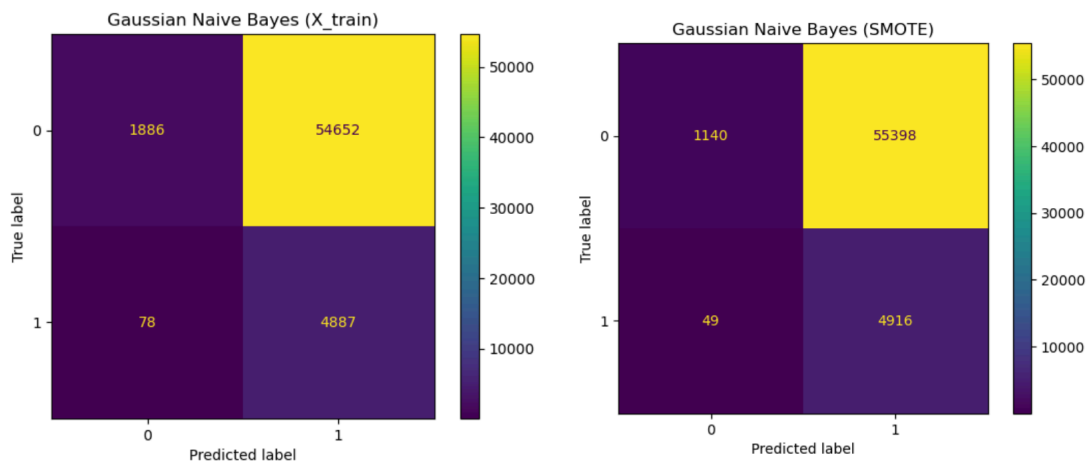
AdaBoost	0	0.0689	0.0711
XGBoost	0.0038	0.0040	0.0018

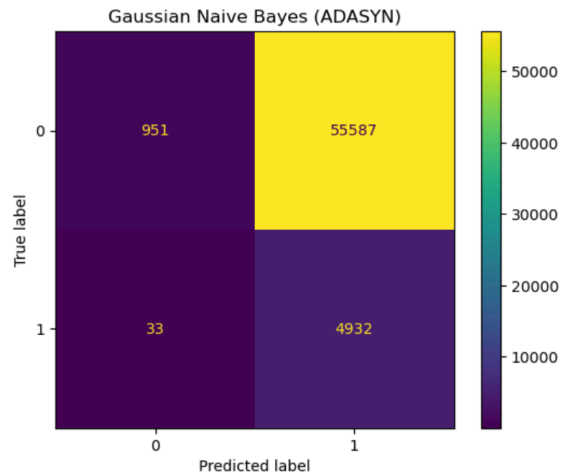
A wide variety of performance is observed. One noticeable theme is that the models trained on SMOTE or ADASYN data almost always perform better than those trained on the original training data. The Gaussian Naive Bayes models performed the best and had particularly impressive recall scores, which is analyzed further later in this section. The logistic regression and AdaBoost models trained on the original training data have 0s across the board; it was determined through analysis of confusion matrices that this was because these models predicted the majority class every time. This is slightly improved when these models are trained on the SMOTE or ADASYN data, but they're still among the weaker models. The XGBoost and Random Forest models also struggled, particularly with recall as they were almost always predicting the majority class. It is somewhat surprising that the Random Forest, AdaBoost, and XGBoost models performed poorly because these ensemble models perform better than the other four relatively simple models in most circumstances. Some hyperparameter tuning using Grid Search with F2 score used to determine the best combination of hyperparameters was conducted for these models using the SMOTE and ADASYN data sets for training to determine if their performance could be improved. This was extremely time consuming, so only 15% of the training data was used in the models generated for each combination of hyperparameters, and it still took several hours to test just a few hundred combinations of hyperparameters. The results are below.

Model (Training data set)	Optimal Hyperparameters	Precision	Recall	F2 Score
Random Forest (SMOTE)	{'max_depth': None, 'min_samples_split': 2, 'n_estimators': 500}	0.2386	0.0518	0.0614
Random Forest (ADASYN)	{'max_depth': None, 'min_samples_split': 2, 'n_estimators': 500}	0.2488	0.0427	0.0512
AdaBoost (SMOTE)	{'learning_rate': 1, 'n_estimators': 300}	0.2587	0.0075	0.0092

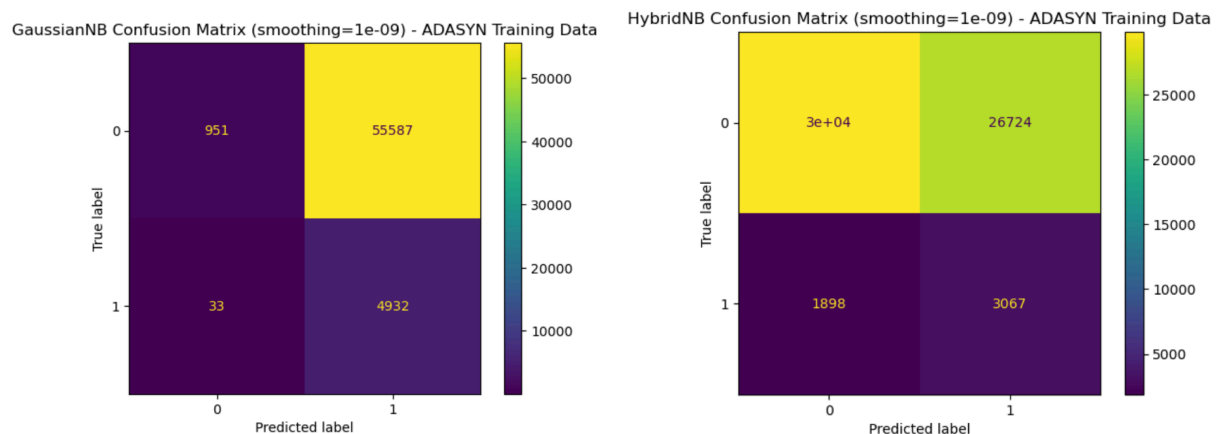
AdaBoost (ADASYN)	{'learning_rate': 1, 'n_estimators': 300}	0.2830	0.0060	0.0075
XGBoost (SMOTE)	{'learning_rate': 0.3, 'max_depth': 5, 'n_estimators': 300}	0.2478	0.0167	0.0205
XGBoost (ADASYN)	{'learning_rate': 0.2, 'max_depth': 10, 'n_estimators': 100}	0.2581	0.0097	0.0120

This does not represent substantial improvement. It is possible that different combinations of hyperparameters or a different classification threshold would result in better performance from these models, but given that the current performance it was decided to further analyze the Gaussian Naive Bayes classifier as it produced by far the best F2 scores. Below are the confusion matrices from these models when trained on the original training data, SMOTE generated training data, and ADASYN generated training data respectively.

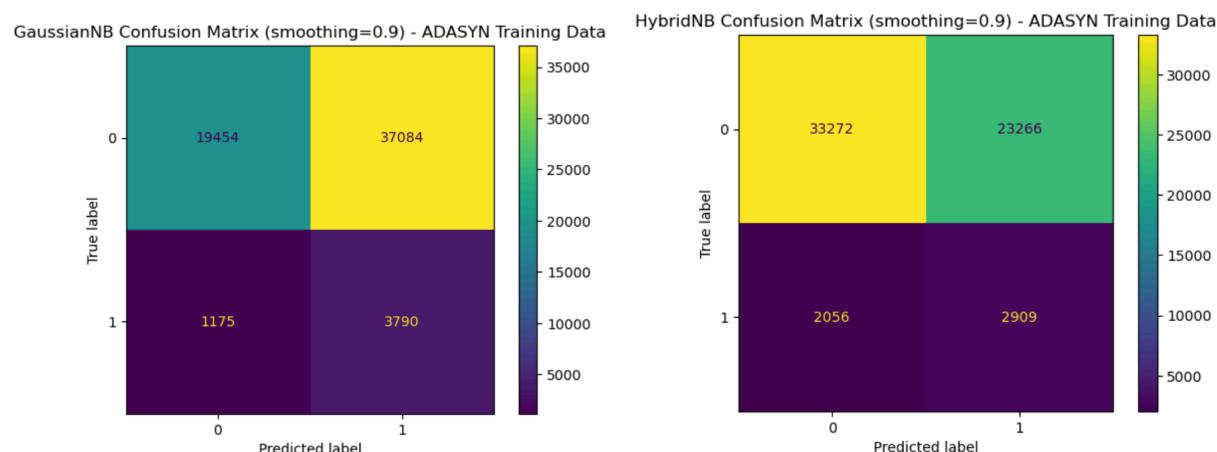




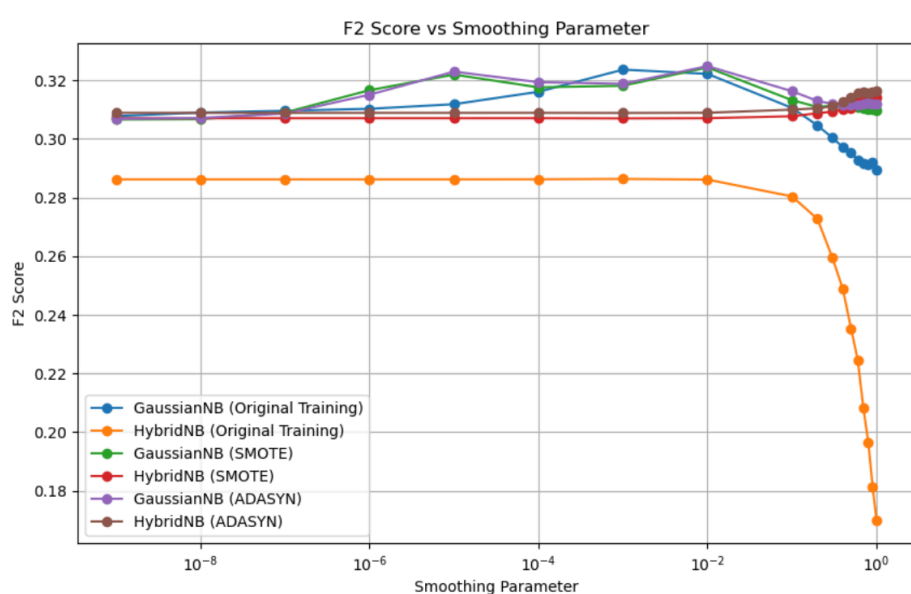
These confusion matrices reflect a very high frequency of predicting that the borrower will default for these models. This does not align with the real world frequency and thus may not be desirable. A few tactics were employed to attempt to train a Naive Bayes classifier that would maintain its relatively high F2 score while not predicting that the borrower will default with such a high frequency. A Hybrid Naive Bayes classifier was trained on the three different training sets, unadjusted, SMOTE generated, and ADASYN generated. The Hybrid Naive Bayes approach involves fitting a Gaussian Naive Bayes classifier using the continuous features, fitting a Bernoulli Naive Bayes classifier using the binary features, multiplying together the likelihoods from each model for each class, then applying Bayes' rule using the resulting likelihoods. These models resulted in similar F2 scores on the test set to the Gaussian Naive Bayes models and had a much more similar distribution of the target predictions to the real values of the target. Below is a comparison of the confusion matrices of the Gaussian Naive Bayes model with default hyperparameters trained using the ADASYN generated data and the Gaussian Naive Bayes model with default hyperparameters trained using the ADASYN generated data to illustrate this. The F2 score of the Gaussian Naive Bayes model shown here is 0.3068 and that of the Hybrid Naive Bayes is 0.3089.

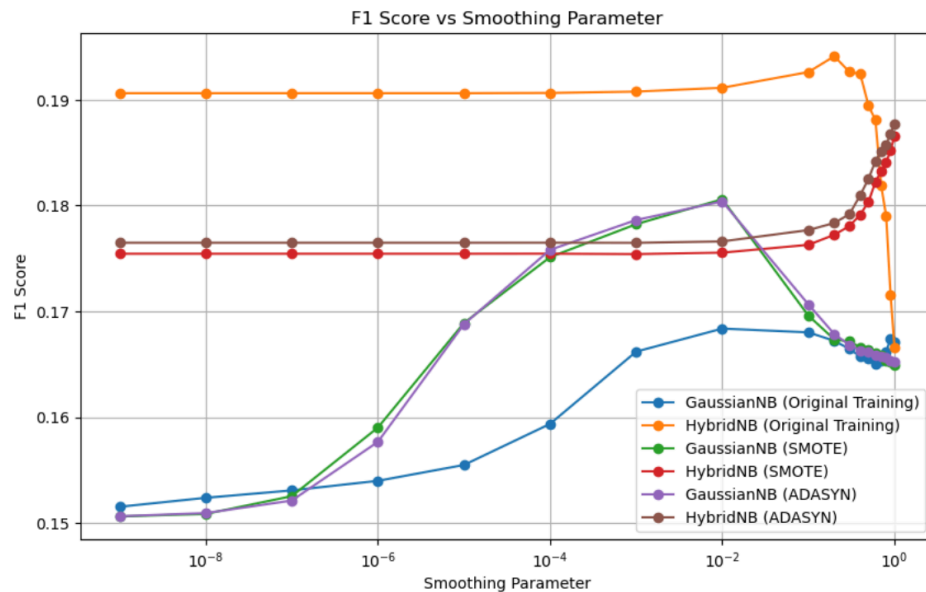


In addition, increasing the smoothing parameter further helped move the distribution of predicted target values closer to that of the actual target values for both the Gaussian Naive Bayes models and the Hybrid Naive Bayes models. Below is a comparison of the Gaussian and Hybrid Naive Bayes models trained on the ADASYN generated data using a higher smoothing parameter than the default. This smoothing parameter represents added variance that makes the model less confident in its predictions. Both predict no default much more frequently than their counterparts with a lower smoothing parameter shown in the previous figure with similar F2 scores, 0.3120 for the Gaussian and 0.3160 for the Hybrid.



Below are plots of the F2 and F1 scores respectively of all of the models with varying values of the smoothing parameter.





Both of these plots reflect that the models trained on SMOTE or ADASYN data remain strong as the smoothing parameter is increased while those trained on the original training data fall in both F2 and F1 score as the smoothing parameter approaches one, which represents another possible advantage of training models on the SMOTE and ADASYN data on top of their already superior performance in most cases.

Conclusion

In this project we determined that Naive Bayes classifiers perform the best of the models that were explored. It initially did so because of a disproportionately high predicted default rate, but the predicted default rate was decreased while maintaining similar F2 scores by using hybrid approach and larger smoothing parameter. We had hoped that the models would be more predictive but the ones we trained are strong relative to those of the notebooks published on Kaggle using this data set. We found that more complex models that perform better in most circumstances struggled and this was not helped by hyperparameter tuning. We also found that models trained on SMOTE and ADASYN data performed better in terms of F2 score than those trained on the original data in the vast majority of circumstances. We believe the techniques shown in this project can be very useful for predicting credit defaults.

There are a number of possible ways to extend and improve this project. The route that we believe has the most potential to improve the predictive power of these models is examining different classification thresholds. We zeroed in on the Naive Bayes classifier for which this is not relevant but for other models there could increase their F2 scores dramatically given that many of the ones we tested were almost never predicting the minority class. In addition, the project could be furthered by spending more time tuning hyperparameters or examining different feature engineering techniques for text based features. Finally, though our purpose was prediction rather than inference, one could use some of the models that we tested to generate

feature importances and draw conclusions about how each feature contributes to credit worthiness.

References

Dataset: <https://www.kaggle.com/datasets/mishra5001/credit-card/data>